Architecture Justification – Campus Placement & Internship Pipeline

Scope: Epic 4, Offer Management & Data Security (FR-004, NFR-001)

Architecture Selection: We chose Microservices Architecture for the Campus Placement & Internship Pipeline System.

Architectural Choice: Microservices with six independently deployable components: Placement Officer Portal, Offer Manager Service, Offer Letter Generator, Notification Service, Encryption Service and Offer Repository, connected through five interfaces (IOfferService, IOfferLetter, IEncryption, IOfferStorage, INotification).

Reason 1: Generating the offer PDF, emailing it, encrypting it and storing it are separate concerns with different failure profiles. As independent services, an outage at the email provider cannot block letter generation or storage, and the Notification Service can retry on its own. This supports FR-004, which requires the offer to reach the student within 10 minutes of selection.

Reason 2: Offers are issued in bursts right after final interview rounds, and NFR-002 requires 500 concurrent students. Microservices let us scale only the busy components (Letter Generator, Notification Service) instead of replicating the whole system, whereas a Layered or Client-Server design scales as one unit and leaves a single point of failure.

Security Advantage: The Encryption Service and Offer Repository sit in an isolated Secure Data Zone. Only the Encryption Service holds the AES-256 keys and may reach the database, so every other component handles offer documents only through the IEncryption interface and never reads stored data directly. This enforces NFR-001 (AES-256 at rest) and limits the damage if another service is compromised. Service-to-service calls use TLS.

Performance Benefit: PDF generation, encryption and email delivery are slow, I/O-heavy tasks. Running them as separate services lets the Offer Manager hand them off without blocking the Placement Officer Portal, keeping page response times under 3 seconds (NFR-002), and each service can be tuned and scaled independently.

Trade-off: Microservices add operational complexity and network latency compared with a Layered design, which we accept in return for fault isolation, independent scaling and a protected data layer.